

Institute of Technology, Nirma University,
Ahmedabad, 382481

Subject:-

Physical Design for CMOS Technology:3EC610IE24

Topic:-

RTL TO GDS IMPLEMENTATION OF
ASYNCHRONOUS FIFO



Submitted By:-

Vivek Singh

23BEI073

Abstract

This project presents the complete RTL-to-GDSII implementation flow of a digital design using industry-standard EDA tools including Genus for synthesis, Innovus for physical design, and NC Launch for simulation and verification. The design flow begins with RTL description in Verilog, followed by logic synthesis, floorplanning, placement, clock tree synthesis (CTS), routing, and finally GDSII generation.

The objective is to understand the backend ASIC design flow with emphasis on timing closure, power optimization, and layout realization. Various constraints such as timing, fanout, and clock definitions are applied using SDC files. Post-route analysis is performed to evaluate setup and hold timing, power consumption, and signal integrity.

The results demonstrate successful implementation of a synthesizable design meeting timing constraints with optimized area and power. This project highlights practical challenges in VLSI physical design and provides insights into real-world ASIC design methodologies.

Table of Contents:

1. Introduction

- 1.1 Background
- 1.2 Objectives of the Assignment
- 1.3 Scope and Limitations

2. Theoretical Background

- 2.1 RTL to GDS Flow
- 2.2 Logic Synthesis
- 2.3 Physical Design Stages
- 2.4 Design Challenges and Complexity in Asynchronous FIFO

3. Methodology / Approach

- 3.1 Tools Used
- 3.2 Design Flow Steps

4. Implementation and Design

- 4.1 Verilog Code
- 4.2 Verilog Testbench
- 4.3 Run.tcl
- 4.4 SDC constraints file
- 4.5 Default.global file
- 4.6 Default.view file

5. Results and Analysis

6. Conclusion

1. Introduction:

1.1 Background:

In modern VLSI design, transforming a high-level hardware description into a manufacturable chip requires a systematic design flow known as the RTL-to-GDSII flow. This flow bridges the gap between functional design and physical realization.

With increasing complexity of integrated circuits, automated tools such as Genus and Innovus are used to ensure optimized performance in terms of speed, power, and area. The flow involves multiple stages including synthesis, placement, clock optimization, and routing.

1.2 Objectives of the Assignment:

The main objectives of this project are:

- To understand the complete RTL-to-GDS design flow
- To implement synthesis using Genus
- To perform physical design using Innovus
- To apply timing constraints using SDC
- To achieve timing closure and generate GDSII output

1.3 Scope and Limitations:

Limitations include:

- No actual fabrication (only GDS generation)
- Limited Libraries available as for academic purpose
- No support of I/O Pad frame libraries

2. Theoretical Background:

2.1 RTL to GDS Flow

The RTL-to-GDS flow consists of:

- RTL Design (Verilog)
- Functional Simulation
- Logic Synthesis
- Floorplanning
- Placement
- Clock Tree Synthesis (CTS)
- Routing
- Physical Verification
- GDSII Generation

2.2 *Logic Synthesis*

Logic synthesis converts RTL code into a gate-level netlist using standard cell libraries.

Key goals:

- Meet timing constraints
- Optimize area
- Reduce power

Constraints are defined using SDC (Synopsys Design Constraints).

2.3 *Physical Design Stages*

- ***Floorplanning:*** *Defines chip layout and macro placement*
- ***Placement:*** *Positions standard cells*
- ***CTS:*** *Balances clock delays*

- **Routing:** *Connects all components*
- **Signoff:** *Ensures design meets all constraints*

2.4 Design Challenges and Complexity in Asynchronous FIFO

Designing an asynchronous FIFO is significantly more complex than a synchronous FIFO due to the presence of multiple clock domains. Since the write and read operations are controlled by independent clocks, several challenges arise in ensuring reliable data transfer without errors.

1. Clock Domain Crossing (CDC)

One of the primary challenges is handling clock domain crossing. Signals generated in one clock domain must be safely transferred to another without causing instability. Directly passing signals between clock domains can lead to incorrect data sampling and unpredictable behaviour.

2. Metastability Issues

When a signal changes near the clock edge of another domain, flip-flops may enter a metastable state where the output is neither logic '0' nor '1' for a short duration. This can propagate errors in the system.

To reduce metastability:

- Multi-stage synchronizers (usually 2 flip-flops) are used
- This allows sufficient time for the signal to stabilize

3. Pointer Synchronization

The FIFO uses read and write pointers to track memory locations. Since these pointers operate in different clock domains, they must be synchronized before comparison.

Challenges include:

- Incorrect pointer comparison due to unsynchronized values

- Risk of false FULL or EMPTY signals

Solution:

- Use synchronized versions of pointers across domains

4. Gray Code Implementation

Binary counters are not suitable for CDC because multiple bits may change simultaneously, increasing the chance of error during sampling.

To overcome this:

- Gray code is used, where only one bit changes at a time
- This reduces the probability of incorrect sampling

5. Full and Empty Condition Detection

Detecting FULL and EMPTY conditions in asynchronous FIFO is complex because:

- Comparison is done between pointers from different domains
- Small synchronization errors can lead to incorrect status flags

Special logic is used:

- FULL condition uses inverted MSB comparison
- EMPTY condition uses direct equality check

6. Timing Constraints and Analysis

Proper timing constraints are required to:

- Ignore false paths between clock domains
- Apply asynchronous clock grouping

Without correct constraints:

- Timing violations may be incorrectly reported
- Design may fail during synthesis or implementation

7. Increased Design Complexity

Compared to synchronous FIFO, asynchronous FIFO:

- Requires additional synchronizer circuits
- Needs careful CDC handling
- Involves complex verification

This increases:

- Design effort
- Verification complexity
- Risk of hidden bugs

3. Methodology / Approach:

3.1 Tools Used

1. Genus (Cadence):

Used for RTL synthesis and netlist generation

2. Innovus (Cadence):

Used for physical design including placement, CTS, and routing

3. NC Launch:

Used for simulation and verification

4. Technology Library:

Standard cell libraries for synthesis and layout

3.2 Procedure / Steps Followed

Step 1: RTL Design

- Verilog code written and verified
- Functional simulation performed

Step 2: Synthesis (Genus)

- RTL compiled
- Constraints applied (clock, timing, fanout)
- Gate-level netlist generated

Step 3: Floorplanning (Innovus)

- Core area defined
- IO placement done
- Power planning performed

Step 4: Placement

- Standard cells placed
- Optimization for congestion and timing

Step 5: Clock Tree Synthesis (CTS)

- Clock buffers inserted
- Skew minimized
- Clock latency balanced

Step 6: Routing

- Global and detailed routing
- Signal connections completed

Step 7: Signoff

- Timing analysis (setup & hold)
- Power analysis
- DRC/LVS checks

Step 8: GDSII Generation

- Final layout exported for fabrication

4. Implementation and Design:

4.1 Verilog Code

```
`timescale 1ns / 1ps
module async_fifo #(
    parameter DATA_WIDTH = 8,
    parameter ADDR_WIDTH = 3
)(
    input wire          wclk,
    input wire          wrst_n,
    input wire          w_en,
    input wire [DATA_WIDTH-1:0] wdata,

    input wire          rclk,
    input wire          rrst_n,
    input wire          r_en,
    output reg [DATA_WIDTH-1:0] rdata,

    output reg          full,
    output reg          empty
);

    reg [DATA_WIDTH-1:0] mem [0:(1<<ADDR_WIDTH)-1];

    reg [ADDR_WIDTH:0] wbin, rbin;
    reg [ADDR_WIDTH:0] wgray, rgray;
```

```
(* ASYNC_REG = "TRUE" *) reg [ADDR_WIDTH:0] wgray_rclk_ff1,  
wgray_rclk_ff2;
```

```
(* ASYNC_REG = "TRUE" *) reg [ADDR_WIDTH:0] rgray_wclk_ff1,  
rgray_wclk_ff2;
```

```
wire [ADDR_WIDTH:0] wgray_sync = wgray_rclk_ff2;
```

```
wire [ADDR_WIDTH:0] rgray_sync = rgray_wclk_ff2;
```

```
wire winc = w_en && !full;
```

```
wire [ADDR_WIDTH:0] wbin_next = wbin + winc;
```

```
wire [ADDR_WIDTH:0] wgray_next = (wbin_next >> 1) ^ wbin_next;
```

```
wire full_next = (wgray_next == {  
    ~rgray_sync[ADDR_WIDTH:ADDR_WIDTH-1],  
    rgray_sync[ADDR_WIDTH-2:0]  
});
```

```
always @(posedge wclk or negedge wrst_n) begin
```

```
    if (!wrst_n) begin
```

```
        wbin <= 0;
```

```
        wgray <= 0;
```

```
        full <= 1'b0;
```

```
    end else begin
```

```
        wbin <= wbin_next;
```

```
        wgray <= wgray_next;
```

```
        full <= full_next;
```

```
        if (winc)
```

```
            mem[wbin[ADDR_WIDTH-1:0]] <= wdata;
```

```

    end
end

wire rinc = r_en && !empty;
wire [ADDR_WIDTH:0] rbin_next = rbin + rinc;
wire [ADDR_WIDTH:0] rgray_next = (rbin_next >> 1) ^ rbin_next;

wire empty_next = (rgray_next == wgray_sync);

always @(posedge rclk or negedge rst_n) begin
    if (!rst_n) begin
        rbin <= 0;
        rgray <= 0;
        empty <= 1'b1;
        rdata <= 0;
    end else begin
        rbin <= rbin_next;
        rgray <= rgray_next;
        empty <= empty_next;
        if (rinc)
            rdata <= mem[rbin[ADDR_WIDTH-1:0]];
    end
end

always @(posedge rclk or negedge rst_n) begin
    if (!rst_n) begin
        wgray_rclk_ff1 <= 0;

```

```

        wgray_rclk_ff2 <= 0;
    end else begin
        wgray_rclk_ff1 <= wgray;
        wgray_rclk_ff2 <= wgray_rclk_ff1;
    end
end

always @(posedge wclk or negedge wrst_n) begin
    if (!wrst_n) begin
        rgray_wclk_ff1 <= 0;
        rgray_wclk_ff2 <= 0;
    end else begin
        rgray_wclk_ff1 <= rgray;
        rgray_wclk_ff2 <= rgray_wclk_ff1;
    end
end

endmodule

```

4.2 Verilog testbench Code

```

`timescale 1ns / 1ps

module tb_async_fifo;

    parameter DATA_WIDTH = 8;

```

```
parameter ADDR_WIDTH = 3;
```

```
reg wclk;
```

```
reg rclk;
```

```
reg wrst_n;
```

```
reg rrst_n;
```

```
reg w_en;
```

```
reg r_en;
```

```
reg [DATA_WIDTH-1:0] wdata;
```

```
wire [DATA_WIDTH-1:0] rdata;
```

```
wire full;
```

```
wire empty;
```

```
async_fifo #(
```

```
    .DATA_WIDTH(DATA_WIDTH),
```

```
    .ADDR_WIDTH(ADDR_WIDTH)
```

```
) dut (
```

```
    .wclk  (wclk),
```

```
    .wrst_n (wrst_n),
```

```
    .w_en   (w_en),
```

```
    .wdata  (wdata),
```

```
    .rclk   (rclk),
```

```
    .rrst_n (rrst_n),
```

```
    .r_en   (r_en),
```

```

        .rdata (rdata),

        .full (full),
        .empty (empty)
    );

    always #5 wclk = ~wclk;
    always #7 rclk = ~rclk;

    initial begin
        wclk = 0;
        rclk = 0;
        wrst_n = 0;
        rrst_n = 0;
        w_en = 0;
        r_en = 0;
        wdata = 8'h00;

        #40;
        wrst_n = 1;
        rrst_n = 1;

        repeat (4) begin
            @(posedge wclk);
            w_en <= 1;
            wdata <= wdata + 8'h11;
            @(posedge wclk);

```

```
    w_en <= 0;  
end
```

```
repeat (2) begin  
    @(posedge rclk);  
    r_en <= 1;  
    @(posedge rclk);  
    r_en <= 0;  
end
```

```
repeat (10) begin  
    @(posedge wclk);  
    if (!full) begin  
        w_en <= 1;  
        wdata <= wdata + 8'h01;  
    end  
    @(posedge wclk);  
    w_en <= 0;  
end
```

```
repeat (10) begin  
    @(posedge rclk);  
    if (!empty)  
        r_en <= 1;  
    @(posedge rclk);  
    r_en <= 0;  
end
```



```

repeat (8) begin
    fork
        begin
            @(posedge wclk);
            if (!full) begin
                w_en <= 1;
                wdata <= wdata + 8'h05;
            end
            @(posedge wclk);
            w_en <= 0;
        end
    begin
        @(posedge rclk);
        if (!empty)
            r_en <= 1;
        @(posedge rclk);
        r_en <= 0;
    end
end
join
end

```

```

repeat (2) begin
    @(posedge wclk);
    if (!full) begin
        w_en <= 1;
        wdata <= wdata + 8'hAA;
    end
end

```

```

        end

        @(posedge wclk);
        w_en <= 0;
    end

    repeat (2) begin
        @(posedge rclk);
        if (!empty)
            r_en <= 1;
        @(posedge rclk);
        r_en <= 0;
    end

    #100;
    $finish;
end

```

endmodule

4.3 run.tcl file

```
set DESIGN_NAME async_fifo
```

```
set_db lib_search_path {/home/install/FOUNDRY/digital/90nm/dig/lib}
```

```
set_db init_hdl_search_path ./
```

```
set_db library {slow.lib typical.lib fast.lib}
```

```
set_db auto_ungroup none
```

```
create_library_set -name slow_lib -timing {slow.lib}
```

```
create_library_set -name fast_lib -timing {fast.lib}
```

```
create_timing_condition -name slow_tc -library_sets {slow_lib}
```

```
create_timing_condition -name fast_tc -library_sets {fast_lib}
```

```
create_delay_corner -name slow_corner -timing_condition slow_tc
```

```
create_delay_corner -name fast_corner -timing_condition fast_tc
```

```
create_constraint_mode -name func_mode -sdc_files {constraints_top.sdc}
```

```
create_analysis_view -name setup_view \
```

```
    -constraint_mode func_mode \
```

```
    -delay_corner slow_corner
```

```
create_analysis_view -name hold_view \
```

```
    -constraint_mode func_mode \
```

```
    -delay_corner fast_corner
```

```
set_analysis_view -setup {setup_view} -hold {hold_view}
```

```
read_hdl async_fifo.v
```

```
elaborate $DESIGN_NAME
```

```
check_design -unresolved
```

```
check_design -multiple_driver
```

```
report_hierarchy > hierarchy.rep
```

```
read_sdc constraints_top.sdc
```

```
report_clocks > clocks.rep
```

```
check_timing > timing_check.rep
```

```
report_area > area.rep
```

```
report_power > power.rep
```

```
set_db syn_global_effort high
```

```
set_db syn_generic_effort high
```

```
set_db syn_map_effort high
```

```
set_max_fanout 20 [current_design]
```

```
set_max_transition 0.3 [current_design]
```

```
redirect compile.log {
```

```
    syn_gen
```

```
    syn_map
```

```
    syn_opt
```

```
}
```

```
write_db ${DESIGN_NAME}_post_syn.db
```

```
report_qor > ${DESIGN_NAME}_qor.rep
```

```
report_timing -group wclk -max_paths 10 >  
${DESIGN_NAME}_wclk_setup.rep
```

```
report_timing -group rclk -max_paths 10 > ${DESIGN_NAME}_rclk_setup.rep
```

```
report_timing -group wclk -nworst 10 -path_type full >  
${DESIGN_NAME}_wclk_hold.rep
```

```
report_area -depth 10 > ${DESIGN_NAME}_area_hier.rep
```

```
report_power > ${DESIGN_NAME}_power.rep
```

```
report_timing -lint > constraint_violation.rep
```

```
check_timing > timing_check.rep
```

```
write_hdl > ${DESIGN_NAME}_netlist.v
```

```
write_sdc > ${DESIGN_NAME}_genus_out.sdc
```

```
write_sdf \
```

```
-timescale ns \
```

```
-nonegchecks \
```

```
-recrem split \
```

```
> ${DESIGN_NAME}.sdf
```

```
gui_show
```

4.4 SDC constraint file

```
create_clock -name wclk -period 10 -waveform {0 5} [get_ports wclk]
```

```
create_clock -name rclk -period 14 -waveform {0 7} [get_ports rclk]
```

```
set_clock_latency 0.2 [get_clocks wclk]
```

```
set_clock_latency 0.2 [get_clocks rclk]
```

```
set_clock_transition 0.1 [get_clocks wclk]
```

```
set_clock_transition 0.1 [get_clocks rclk]
```

```
set_clock_uncertainty 0.05 [get_clocks wclk]
```

```
set_clock_uncertainty 0.05 [get_clocks rclk]
```

```
set_clock_groups -asynchronous \
```

```
-group [get_clocks wclk] \
```

```
-group [get_clocks rclk]
```

```
set_false_path -from [get_ports wrst_n]
```

```
set_false_path -from [get_ports rrst_n]
```

```
set_case_analysis 1 [get_ports wrst_n]
```

```
set_case_analysis 1 [get_ports rrst_n]
```

```
set_input_delay -max 1.0 -clock wclk [get_ports w_en]
```

```
set_input_delay -min 0.2 -clock wclk [get_ports w_en]
```

```
set_input_delay -max 1.0 -clock wclk [get_ports {wdata[*]}]
```

```
set_input_delay -min 0.2 -clock wclk [get_ports {wdata[*]}]
```

```
set_input_delay -max 1.0 -clock rclk [get_ports r_en]
```

```
set_input_delay -min 0.2 -clock rclk [get_ports r_en]
```

```
set_output_delay -max 1.0 -clock rclk [get_ports {rdata[*]}]
```

```
set_output_delay -min 0.2 -clock rclk [get_ports {rdata[*]}]
```

```
set_output_delay -max 1.0 -clock wclk [get_ports full]
```

```
set_output_delay -min 0.2 -clock wclk [get_ports full]
```

```
set_output_delay -max 1.0 -clock rclk [get_ports empty]
```

```
set_output_delay -min 0.2 -clock rclk [get_ports empty]
```

```
set_max_delay 0.5 \
```

```
-from [get_cells *ff1*] \
```

```
-to [get_cells *ff2*]
```

```
set_max_transition 0.3 [current_design]
```

```
set_max_capacitance 0.2 [current_design]
```

```
set_timing_derate -late 1.05
```

```
set_timing_derate -early 0.95
```

4.5 Default.global file

```
set defHierChar {/}
```

```
set init_top_cell async_fifo
```

```
set init_pwr_net {VDD}
```

```
set init_gnd_net {VSS}
```

```
set init_pwr_port VDD
```

```
set init_gnd_port VSS
```

```
set                                                                    init_lef_file  
{/home/install/FOUNDRY/digital/90nm/dig/lef/gsclib090_translated.lef  
/home/install/FOUNDRY/digital/90nm/dig/lef/gsclib090_macro.lef  
/home/install/FOUNDRY/digital/90nm/dig/lef/gsclib090_tech.lef}
```

```
set init_verilog {async_fifo_netlist.v}
```

```
set init_mmmc_file {Default.view}
```

```
set init_design_settop 1
```

```
set init_design_uniquify 1
```

```
set distributed_client_message_echo 1
```

```
set eco_post_client_restore_command {update_timing ; write_eco_opt_db ;}
```


4.6 Default.view file

```
create_library_set -name LIB_SS \  
-timing {/home/install/FOUNDRY/digital/90nm/dig/lib/slow.lib}
```

```
create_library_set -name LIB_FF \  
-timing {/home/install/FOUNDRY/digital/90nm/dig/lib/fast.lib}
```

```
create_library_set -name LIB_TT \  
-timing {/home/install/FOUNDRY/digital/90nm/dig/lib/typical.lib}
```

```
create_rc_corner -name RC_BEST  
create_rc_corner -name RC_TYP  
create_rc_corner -name RC_WORST
```

```
create_constraint_mode -name FUNC_MODE \  
-sdc_files {constraints_top.sdc}
```

```
create_delay_corner -name DC_SETUP \  
-library_set {LIB_SS} \  
-rc_corner {RC_WORST}
```

```
create_delay_corner -name DC_HOLD \  
-library_set {LIB_FF} \  
-rc_corner {RC_BEST}
```

```
create_delay_corner -name DC_TYP \  

```

-library_set {LIB_TT} \

-rc_corner {RC_TYP}

create_analysis_view -name SETUP_VIEW \

-constraint_mode {FUNC_MODE} \

-delay_corner {DC_SETUP}

create_analysis_view -name HOLD_VIEW \

-constraint_mode {FUNC_MODE} \

-delay_corner {DC_HOLD}

create_analysis_view -name TYP_VIEW \

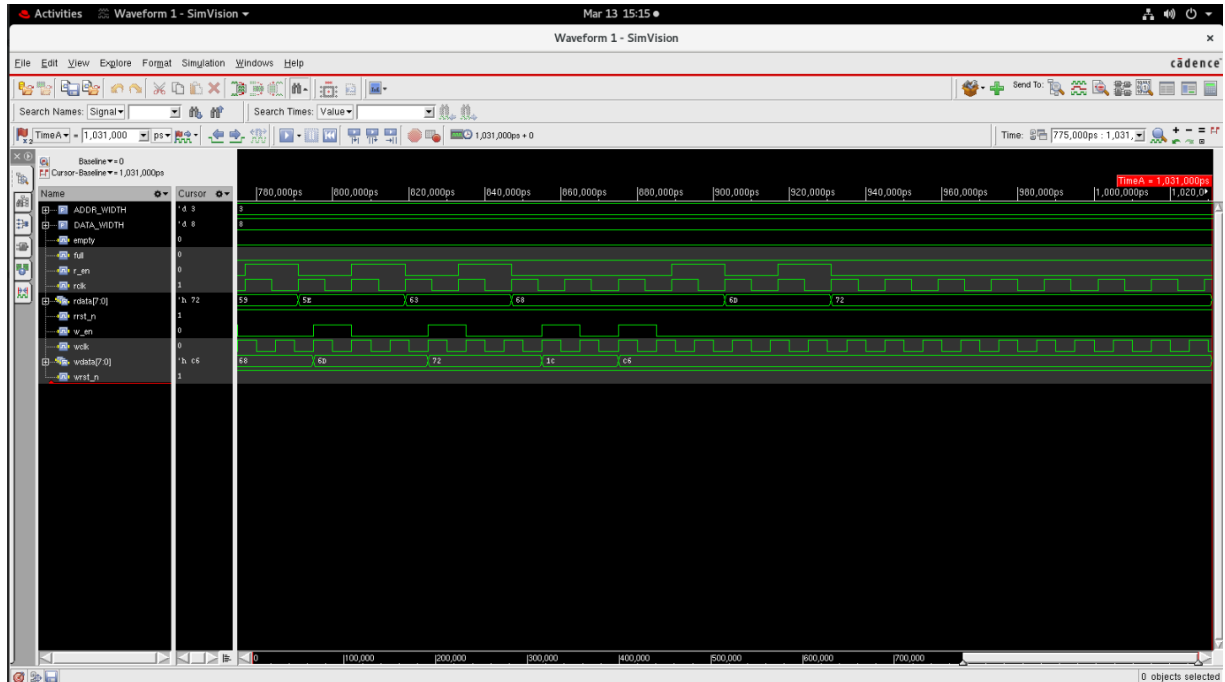
-constraint_mode {FUNC_MODE} \

-delay_corner {DC_TYP}

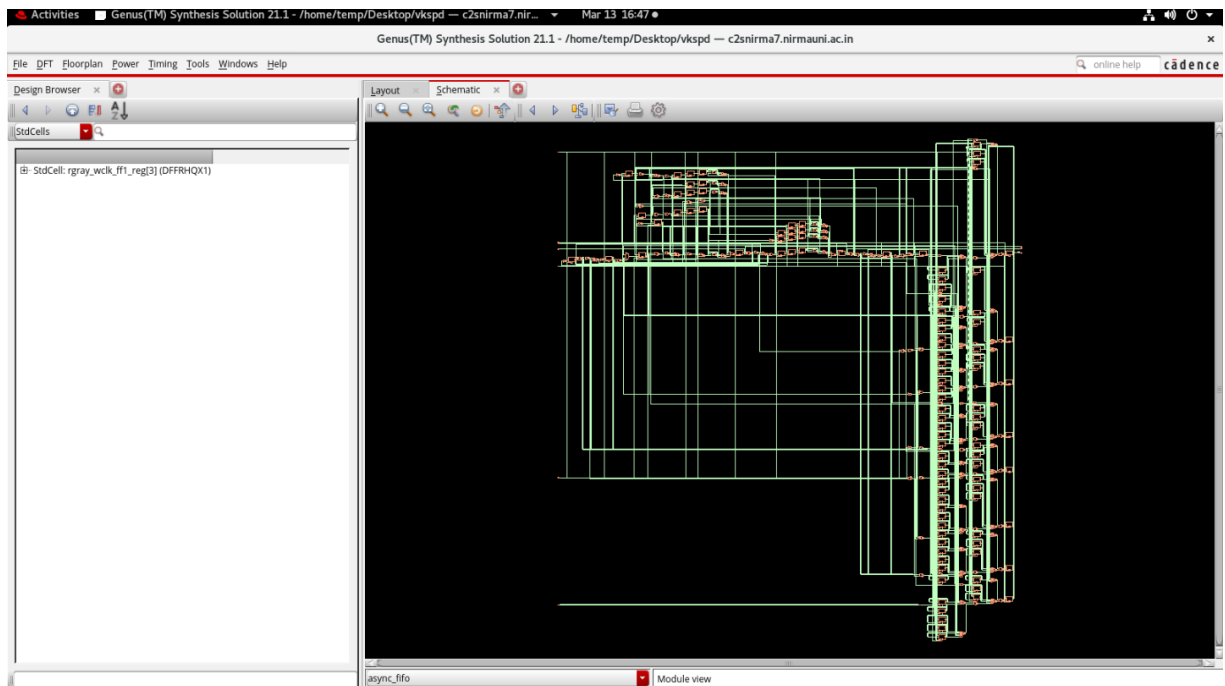
set_analysis_view -setup {SETUP_VIEW} -hold {HOLD_VIEW}

5.Results and Analysis:

5.1 Pre layout Simulation



5.2 Genus Synthesis



5.3 Power report

Instance: /async_fifo
Power Unit: W
PDB Frames: /stim#0/frame#0

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
register	1.36094e-05	1.78985e-04	3.03561e-06	1.95630e-04	86.58%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	2.24626e-06	9.67400e-06	5.12853e-06	1.70488e-05	7.55%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	0.00000e+00	0.00000e+00	1.32701e-05	1.32701e-05	5.87%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	1.58557e-05	1.88659e-04	2.14343e-05	2.25948e-04	100.00%
Percentage	7.02%	83.50%	9.49%	100.00%	100.00%

5.4 Timing Report

5.4.1 Pre CTS

The screenshot displays the Cadence Innovus implementation system interface. The terminal window on the left shows the results of a timing analysis for the 'async_fifo' instance. The reports indicate that the design is meeting timing requirements, with no violations found. The floorplan window on the right shows a detailed view of the circuit layout, including the placement of components and the routing of signals.

Terminal Output:

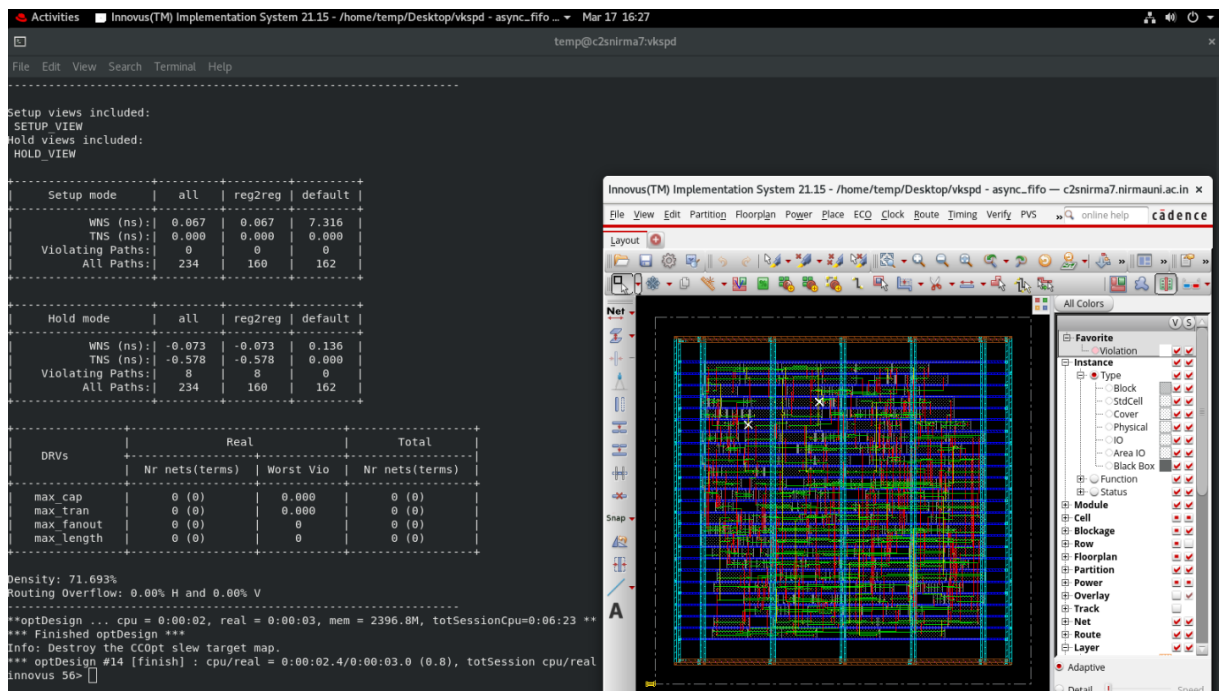
```
*** Message Summary: 40 warning(s), 0 error(s)
*** place opt design #3 [finish] : cpu/real = 0:00:04.2/0:00:05.3 (0.8), totSession cpu/real = 0:02:03.8/0:23:18.4 (0.1), mem = 1906.0M
binnovus 3> *** time design #4 [begin] : totSession cpu/real = 0:02:04.2/0:23:24.0 (0.1), mem = 1906.0M
start to check current routing status for nets...
All nets are already routed correctly.
end to check current routing status for nets (mem=1906.0M)

time design Summary
-----
Setup views included:
SETUP VIEW
-----
Setup mode | all | reg2reg | default |
WNS (ns) | 0.053 | 0.053 | 7.232 |
TNS (ns) | 0.000 | 0.000 | 0.000 |
Violating Paths | 0 | 0 | 0 |
All Paths | 234 | 160 | 162 |

-----
DRVs | Real | Total |
-----
max_cap | 0 (0) | 0.000 | 0 (0) |
max_tran | 0 (0) | 0.000 | 0 (0) |
max_fanout | 0 (0) | 0 | 0 (0) |
max_length | 0 (0) | 0 | 0 (0) |

Density: 69.731%
Routing Overflow: 0.00% H and 0.00% V
Reported timing to dir timingReports
Total CPU time: 0.1 sec
Total Real time: 1.0 sec
Total Memory Usage: 1906.210938 Mbytes
*** time design #4 [finish] : cpu/real = 0:00:00.1/0:00:00.9 (0.1), totSession cp
binnovus 3>
```

5.4.2 Post CTS



Activities | Innovus(TM) Implementation System 21.15 - /home/temp/Desktop/vkspd - async_fifo... - Mar 17 16:27

temp@c2snirma7:vkspd

Setup views included:
SETUP_VIEW

Hold views included:
HOLD_VIEW

Setup mode	all	reg2reg	default
WNS (ns):	0.067	0.067	7.316
TNS (ns):	0.000	0.000	0.000
Violating Paths:	0	0	0
All Paths:	234	160	162

Hold mode	all	reg2reg	default
WNS (ns):	-0.073	-0.073	0.136
TNS (ns):	-0.578	-0.578	0.000
Violating Paths:	0	0	0
All Paths:	234	160	162

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	0 (0)	0.000	0 (0)
max_tran	0 (0)	0.000	0 (0)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Density: 71.693%

Routing Overflow: 0.00% H and 0.00% V

***optDesign ... cpu = 0:00:02, real = 0:00:03, mem = 2396.8M, totSessionCpu=0:06:23 +*
*** Finished optDesign ***
Info: Destroy the cCopt slew target map.
*** optDesign #14 [finish] : cpu/real = 0:00:02.4/0:00:03.0 (0.8), totSession cpu/real
innovus 56>

Innovus(TM) Implementation System 21.15 - /home/temp/Desktop/vkspd - async_fifo... - c2snirma7.nirmauni.ac.in

File View Edit Partition Floorplan Power Place ECO Clock Route Timing Verify PVS online help

Layout

Net

Favorite Violation

Instance

Type

Block

StdCell

Cover

Physical

IO

Area IO

Black Box

Function

Status

Module

Cell

Blockage

Row

Floorplan

Partition

Power

Overlay

Track

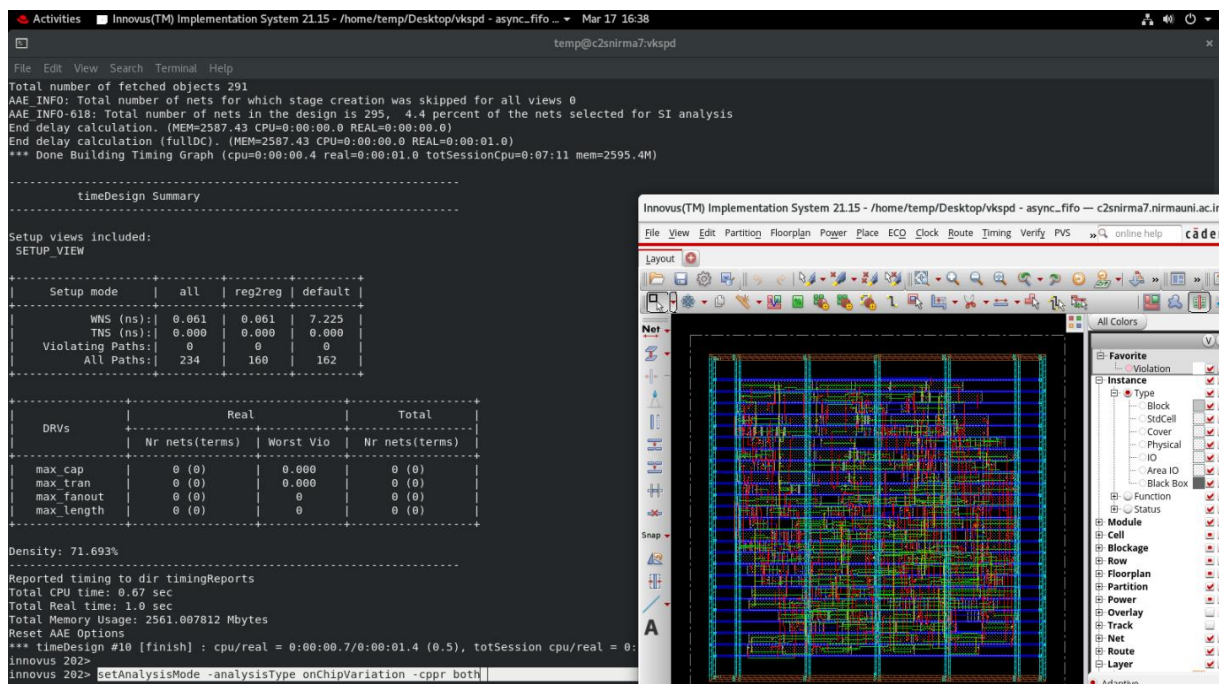
Net

Route

Layer

Adaptive

5.4.3 Post Route



Activities | Innovus(TM) Implementation System 21.15 - /home/temp/Desktop/vkspd - async_fifo... - Mar 17 16:38

temp@c2snirma7:vkspd

Total number of fetched objects 291

AAE INFO: Total number of nets for which stage creation was skipped for all views 0

AAE INFO-618: Total number of nets in the design is 295, 4.4 percent of the nets selected for SI analysis

End delay calculation. (MEM=2587.43 CPU=0:00:00.0 REAL=0:00:00.0)

End delay calculation (fullDC). (MEM=2587.43 CPU=0:00:00.0 REAL=0:00:01.0)

*** Done Building Timing Graph (cpu=0:00:00.4 real=0:00:01.0 totSessionCpu=0:07:11 mem=2595.4M)

timeDesign Summary

Setup views included:
SETUP_VIEW

Setup mode	all	reg2reg	default
WNS (ns):	0.061	0.061	7.225
TNS (ns):	0.000	0.000	0.000
Violating Paths:	0	0	0
All Paths:	234	160	162

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	0 (0)	0.000	0 (0)
max_tran	0 (0)	0.000	0 (0)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Density: 71.693%

Reported timing to dir timingReports

Total CPU time: 0.67 sec

Total Real time: 1.0 sec

Total Memory Usage: 2561.007812 Mbytes

Reset AAE Options

*** timeDesign #10 [finish] : cpu/real = 0:00:00.7/0:00:01.4 (0.5), totSession cpu/real = 0:
innovus 202>

innovus 202> SetAnalysisMode -analysisType onChipVariation -cpr both

Innovus(TM) Implementation System 21.15 - /home/temp/Desktop/vkspd - async_fifo... - c2snirma7.nirmauni.ac.in

File View Edit Partition Floorplan Power Place ECO Clock Route Timing Verify PVS online help

Layout

Net

Favorite Violation

Instance

Type

Block

StdCell

Cover

Physical

IO

Area IO

Black Box

Function

Status

Module

Cell

Blockage

Row

Floorplan

Partition

Power

Overlay

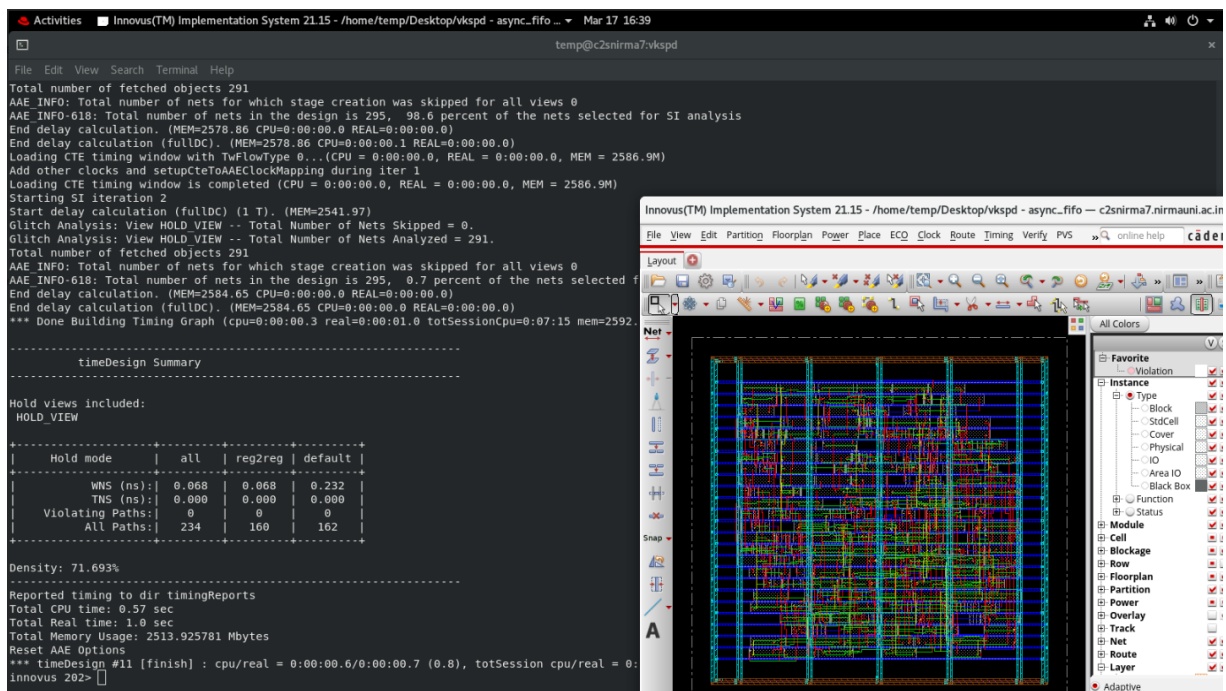
Track

Net

Route

Layer

Adaptive



5.5 Clock report

```

Generated by: Genus(TM) Synthesis Solution 21.14-s082_1
Generated on: Mar 13 2026 04:45:34 pm
Module:      async_fifo
Technology libraries: slow
              typical
              fast
Operating conditions: slow (balanced_tree)
Wireload mode: enclosed
Area mode:   timing library
=====

Clock Description
-----

Clock Name      Period  Rise  Fall      Clock Domain  Source Pin/Port  No of Registers
-----
wclk            10000.0  0.0   5000.0    domain_1      wclk          81
rc1k            14000.0  0.0   7000.0    domain_1      rc1k          25

Clock Network Latency / Setup Uncertainty
-----

Clock Name      Network Latency Rise  Network Latency Fall  Source Latency Rise  Source Latency Fall  Setup Uncertainty Rise  Setup Uncertainty Fall
-----
wclk            200.0      200.0      0.0      0.0      50.0      50.0
rc1k            200.0      200.0      0.0      0.0      50.0      50.0

Clock Relationship (with uncertainty & latency)
-----

From  To      R->R      R->F      F->R      F->F
-----
wclk  wclk    9950.0    4950.0    4950.0    9950.0
wclk  rc1k    1950.0    950.0     950.0     1950.0
rc1k  wclk    950.0     950.0     950.0     1950.0
rc1k  rc1k   13950.0   6950.0    6950.0    13950.0

```

5.6 QoR report

```
=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
Generated on:      Mar 13 2026  04:45:38 pm
Module:            async_fifo
Technology libraries: slow
                  typical
                  fast
Operating conditions: slow (balanced_tree)
Wireload mode:     enclosed
Area mode:         timing library
=====
```

Timing

Clock Period

rclk 14000.0

wclk 10000.0

Cost Group	Critical Path Slack	TNS	Violating Paths
default	No paths	0.0	
rclk	-28.8	-115.2	4
wclk	-28.8	-115.2	4
Total		-230.4	8

Instance Count

Leaf Instance Count	231
Physical Instance count	0
Sequential Instance Count	104
Combinational Instance Count	127
Hierarchical Instance Count	0

Area

Cell Area	2826.265
Physical Cell Area	0.000
Total Cell Area (Cell+Physical)	2826.265
Net Area	0.000
Total Area (Cell+Physical+Net)	2826.265

Max Fanout	80 (wclk)
Min Fanout	0 (rrst_n)
Average Fanout	2.8
Terms to net ratio	3.8952
Terms to instance ratio	4.1818
Runtime	8.318424 seconds
Elapsed Runtime	29 seconds
Genus peak memory usage	1766.39
Innovus peak memory usage	no_value
Hostname	localhost

5.7 Nano route report

```
Activities Terminal Mar 17 16:32
temp@c2snirma7:vkspd

File Edit View Search Terminal Help
#Total wire length on LAYER Metal3 = 2140 um.
#Total wire length on LAYER Metal4 = 318 um.
#Total wire length on LAYER Metal5 = 0 um.
#Total wire length on LAYER Metal6 = 0 um.
#Total wire length on LAYER Metal7 = 0 um.
#Total wire length on LAYER Metal8 = 0 um.
#Total wire length on LAYER Metal9 = 0 um.
#Total number of vias = 1848
#Up-Via Summary (total 1848):
#
#-----
# Metal1      1035
# Metal2       698
# Metal3       115
#-----
#              1848
#
#detailRoute Statistics:
#Cpu time = 00:00:01
#Elapsed time = 00:00:01
#Increased memory = 8.82 (MB)
#Total memory = 2341.72 (MB)
#Peak memory = 2444.34 (MB)
#
# no debugging net set
#
#globalDetailRoute statistics:
#Cpu time = 00:00:02
#Elapsed time = 00:00:02
#Increased memory = 17.03 (MB)
#Total memory = 2304.58 (MB)
#Peak memory = 2444.34 (MB)
#Number of warnings = 9
#Total number of warnings = 26
#Number of fails = 0
#Total number of fails = 0
#Complete globalDetailRoute on Tue Mar 17 16:32:29 2026
#
#Default setup view is reset to SETUP_VIEW.
#Default setup view is reset to SETUP_VIEW.
AAE INFO: Post Route call back at the end of routeDesign
#routeDesign: cpu time = 00:00:02, elapsed time = 00:00:02, memory = 2286.71 (MB), peak = 2444.34 (MB)
*** Message Summary: 0 warning(s), 0 error(s)

innovus 57> |
```

5.8 area.rep

```
=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
Generated on:      Mar 13 2026 04:45:34 pm
Module:            async_fifo
Technology libraries: slow
                   typical
                   fast
Operating conditions: slow (balanced_tree)
Wireload mode:     enclosed
Area mode:          timing library
=====

Instance      Module      Cell Count  Cell Area  Net Area  Total Area  Wireload
-----
async_fifo          201    3909.960    0.000    3909.960 <none> (D)
  g4                not_op        2      7.629    0.000      7.629 <none> (D)
  add_44_43          add_unsigned    13    76.292    0.000    76.292 <none> (D)
  add_70_43          add_unsigned    13    76.292    0.000    76.292 <none> (D)
  mux_mem[rbin]_86_26 bmux_14        8   488.268    0.000   488.268 <none> (D)
  ctl_wbin_62_17     case_box       19   133.511    0.000   133.511 <none> (D)
(D) = wireload is default in technology library
|
```

5.9 No violation report

```
Activities Terminal Mar 17 16:41
temp@c2snirma7:vkspd

File Edit View Search Terminal Help
innovus 203>
innovus 203>
innovus 203>
innovus 203>
innovus 203>
innovus 203> report_constraint -all violators
#####
# Generated by: Cadence Innovus 21.15-s110 1
# OS: Linux x86_64(Host ID c2snirma7.nirmauni.ac.in)
# Generated on: Tue Mar 17 16:41:52 2026
# Design: async_fifo
# Command: report_constraint -all violators
#####
# format : frame 1 : split 1

max_delay/setup
-----
No paths found

Check type : clock_period
-----
No paths found

Check type : skew
-----
No paths found

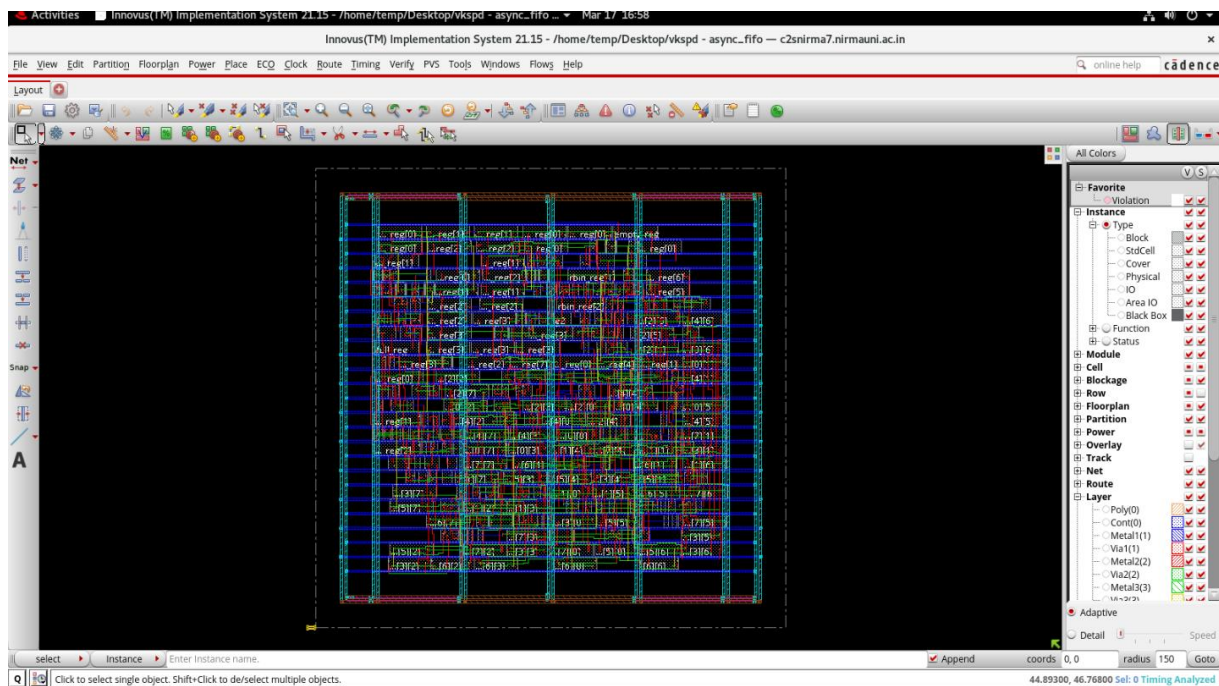
Check type : pulse_width
-----
No violating Checks with given description found

Check type : max_transition
-----
No Violations found

Check type : max_capacitance
-----
No Violations found

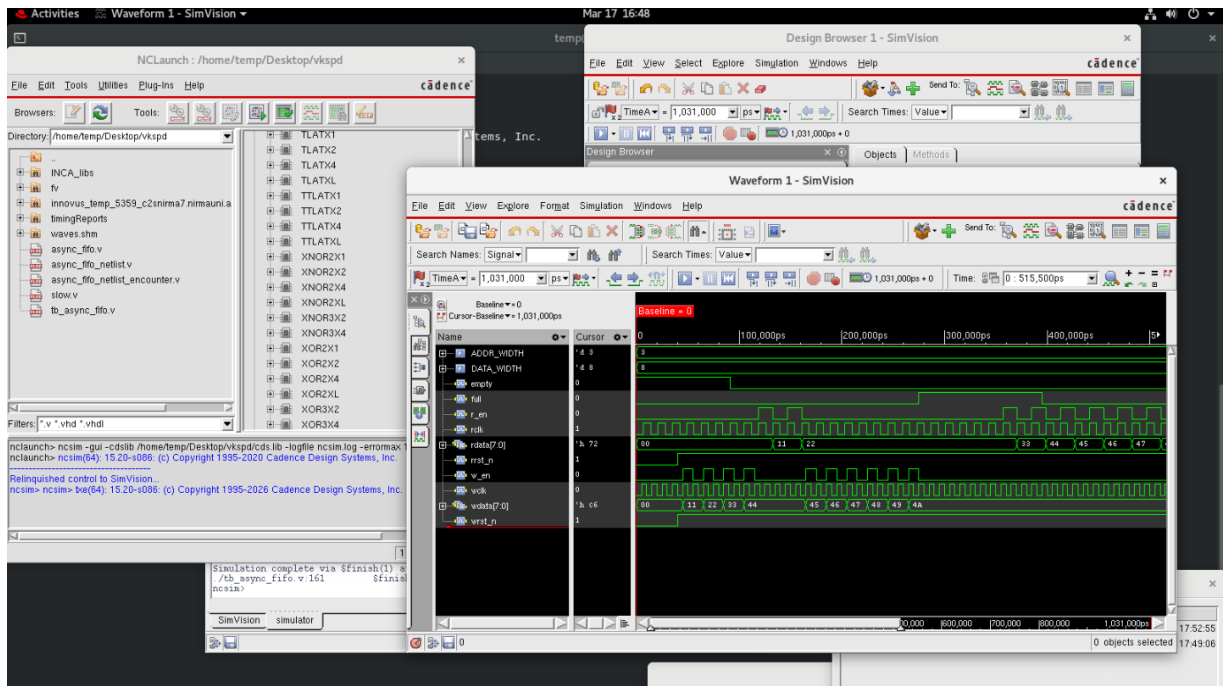
Check type : max_fanout
-----
No Violations found
innovus 204>
```

5.10 Layout





5.11 Post Layout Simulation



6. Conclusion:

The RTL-to-GDSII flow was successfully implemented using Genus and Innovus. The design met timing, area, and power constraints, demonstrating a complete backend ASIC design flow.

The project provided practical exposure to synthesis, placement, clock tree design, and routing. Challenges such as timing violations and routing congestion were resolved using optimization techniques.

This work highlights the importance of constraints, tool settings, and iterative optimization in achieving a manufacturable chip design. Future work can include larger designs, advanced node technologies, and deeper signoff analysis.